

DFS Templates

DFS appears in five forms. The pattern determines naming, return type, and where the answer is built.

When to Use — Signal → Pattern

Problem signal phrase	Pattern / template
"height / depth / sum OF the subtree", combine results upward	#1 Process children first, combine at node (post-order)
"root-to-leaf path", "number built along the path"	#2 Process node first, pass state down (pre-order)
"longest/best path that may bend through a node" (spans both arms)	#3 Tree global answer
"count islands / largest region / flood fill" on a grid	#4 Grid DFS
"can finish all", "detect cycle", "valid course order" (directed)	#5 Graph DFS 3-color
"all subsets / permutations / combinations", "find every way"	#6 Backtracking
"valid BST", "kth smallest", "closest value", "successor"	BST templates (pass bounds / inorder / binary search)

Quick Reference Table

#	Name	Description	Pattern	Returns	Global var
104	Max Depth	Maximum depth of binary tree	Tree — return up	depth int	No
543	Diameter	Longest path between any two nodes	Tree — return up	height int	Yes (diameter)
124	Max Path Sum	Max sum path between any two nodes	Tree — return up	best single-arm int	Yes (maxSum)
110	Balanced Tree	Is tree height-balanced?	Tree — return up	height or -1	No
226	Invert Tree	Mirror a binary tree	Tree — return up	TreeNode	No
112	Path Sum	Any root-to-leaf path summing to target?	Tree — pass down	boolean	No
113	Path Sum II	All root-to-leaf paths summing to target	Tree — pass down + backtrack	void	No
257	Binary Tree Paths	All root-to-leaf paths as strings	Tree — pass down + backtrack	void	No
129	Sum Root to Leaf	Sum of all root-to-leaf numbers	Tree — pass down	int	No
200	Number of Islands	Count connected land regions in grid	Grid DFS	void	No
695	Max Area of Island	Largest island area	Grid DFS	int (area)	No
207	Course Schedule	Can all courses be finished (no cycle)?	Graph DFS 3-color	boolean	No
210	Course Schedule II	Return valid course order	Graph DFS 3-color	void → post-order	No
46	Permutations	All permutations of distinct integers	Backtrack — no start, used[]	void	No
78	Subsets	All subsets of distinct integers	Backtrack — start, no used[]	void	No
39	Combination Sum	All combos summing to target (reuse allowed)	Backtrack — start, reuse i	void	No
40	Combination Sum II	All combos summing to target (no reuse)	Backtrack — start, skip dups	void	No
79	Word Search	Does word exist in board?	Grid backtrack — restore cell	boolean	No
131	Palindrome Partitioning	All palindrome partitions of string	Backtrack — start, check palindrome	void	No
98	Validate Binary Search Tree	Check if a binary tree is a valid BST	Pass min/max bounds down recursively; every node must satisfy both constraints	Pass bounds down: validate(node, min, max) with long bounds to handle INT_MIN/INT_MAX	O(n)
230	Kth Smallest Element in a BST	Find kth smallest element	Inorder traversal (left→node→right) counts ascending; stop at kth	Inorder + counter: global count/result; return early when count == k	O(k)
270	Closest BST Value	Find value in BST closest to target	BST search: go left if target < node, right if target > node; track closest	BST binary search: use BST structure to traverse in O(h)	O(h)
285	Inorder Successor in BST	Find the smallest node greater than p	BST traversal: if root > p it could be successor; else go right	BST successor search: save candidate when going left	O(h)

#	Name	Description	Pattern	Returns	Global var
5 4 9	Binary Tree Longest Consecutive Sequence II	Longest consecutive path (increasing OR decreasing, may pass through node)	Post-order DFS returning {inc, dec} lengths per node	Two lengths returned: track both increasing and decreasing runs; combine at node	O(n)
6 8 7	Longest Univalue Path	Longest path where all nodes have same value	Post-order DFS; at each node, extend path from children only if values match	Conditional extend: leftPath = (left.val==node.val) ? left+1 : 0	O(n)
3 7	Sudoku Solver	Fill 9x9 grid so each row/col/3x3 box has 1-9	Backtracking; try 1-9 in each empty cell, validate, recurse	Constraint validation: check row, col, and 3x3 box before placing	O(9^m)
4 7	Permutations II	All unique permutations of array with duplicates	Backtracking + sort + used[]; skip used[i-1] duplicate	Skip duplicate at same level: if i>0 && nums[i]==nums[i-1] && !used[i-1] continue	O(n·n!)
5 1	N-Queens	All board placements where no queens attack	Backtracking by row; track cols, diag1 (r-c), diag2 (r+c) sets	Diagonal sets: r-c and r+c uniquely identify diagonals	O(n!)
6 0	Permutation Sequence	kth permutation of 1..n (1-indexed)	Math: factorial number system; pick each digit directly	Direct math, no backtracking: index = (k-1)/(n-1)!; remove used digit	O(n²)
9 0	Subsets II	All unique subsets of array with duplicates	Backtracking; sort first; skip i>start && nums[i]==nums[i-1]	Skip duplicate branches: sort + skip same value at same depth	O(n·2^n)
2 1 6	Combination Sum III	k numbers from 1-9 summing to n, no reuse	Backtracking with start index advancing i+1	Two constraints: track both count k and remaining sum	O(C(9,k))

All Templates

```
// 1. PROCESS CHILDREN FIRST, COMBINE AT NODE (post-order)
// MENTAL MODEL: each node trusts its children to return a finished answer, then merges them.
// WHEN: "what is the height/sum/property OF the subtree rooted here"
private int dfs(TreeNode node) {
    if (node == null) return BASE;           // 0 for depth, true for balanced
    int left = dfs(node.left);
    int right = dfs(node.right);
    // combine left, right, node.val → answer propagates up
    return ...;
}

// 2. PROCESS NODE FIRST, PASS STATE DOWN (pre-order)
// MENTAL MODEL: carry what you've seen so far down the path; the leaf has the full picture.
// WHEN: "root-to-leaf path", "running sum/number built along the way"
private void dfs(TreeNode node, int accumulated) {
    if (node == null) return;
    accumulated += node.val;                 // update state on way down
    if (isLeaf(node)) { /* record */ return; }
    dfs(node.left, accumulated);
    dfs(node.right, accumulated);
    // backtrack: no undo needed for primitive; undo List.add if using collection
}

// 3. TREE – GLOBAL ANSWER (return up the best single arm; update global across both arms)
// MENTAL MODEL: the answer may bend through a node using both arms, but only one arm can extend upward.
// WHEN: use when the best path spans across a node, not just up one arm.
int answer = Integer.MIN_VALUE;
private int dfs(TreeNode node) {
    if (node == null) return 0;
    int left = Math.max(0, dfs(node.left)); // clamp negative
    int right = Math.max(0, dfs(node.right));
    answer = Math.max(answer, left + right + node.val); // cross-node path
    return Math.max(left, right) + node.val;          // single arm up
}

// 4. GRID DFS – mark visited by mutating grid; 4-directional flood fill
// MENTAL MODEL: stand on a cell, flood into all 4 neighbors, sinking each as you visit it.
// WHEN: "connected region / island / flood fill" on a grid
private void dfs(int[][] grid, int r, int c) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return;
    if (grid[r][c] != TARGET) return;
    grid[r][c] = VISITED;
    dfs(grid, r+1, c); dfs(grid, r-1, c);
    dfs(grid, r, c+1); dfs(grid, r, c-1);
}

// 5. GRAPH DFS – 3-color visited: 0=unseen 1=in-stack 2=done
// MENTAL MODEL: if you re-enter a node still on the current stack, you've looped back → cycle.
// WHEN: "can finish all", "detect cycle", "valid ordering" on a directed graph
int[] state;
private boolean dfs(int node) {
    if (state[node] == 1) return true;      // back edge → cycle
    if (state[node] == 2) return false;    // already safe
    state[node] = 1;
    for (int neighbor : graph.get(node))
        if (dfs(neighbor)) return true;
    state[node] = 2;
    return false;
}

// 6. BACKTRACKING – choose / recurse / undo
// MENTAL MODEL: build a candidate one element at a time; undo each choice to explore the next branch.
// WHEN: "all subsets / permutations / combinations", "find all ways"
```

```
private void backtrack(int start, List<Integer> current) {
    if (baseCase) { result.add(new ArrayList<>(current)); return; }
    for (int i = start; i < n; i++) {
        if (skipCondition(i)) continue;    // prune duplicates
        current.add(nums[i]);              // choose
        backtrack(next, current);          // next = i (reuse) or i+1 (no reuse)
        current.remove(current.size()-1);  // undo
    }
}
```

Part 1 — Tree DFS

PROCESS CHILDREN FIRST, COMBINE AT NODE (post-order)

Recurse into children first, then combine results at the current node. Answer propagates upward.

#104 Maximum Depth of Binary Tree

Description: Maximum number of nodes along the longest root-to-leaf path.

Intuition: my depth is just one more than my deeper child's depth.

```
class Solution {
    public int maxDepth(TreeNode root) {
        return dfs(root);
    }
    private int dfs(TreeNode node) {
        if (node == null) return 0;
        int left = dfs(node.left);
        int right = dfs(node.right);
        return Math.max(left, right) + 1;
    }
}
```

Time $O(n)$ | **Space** $O(h)$

#543 Diameter of Binary Tree

Description: Length of the longest path between any two nodes (path does not need to pass through root).

Key: the path through a node = `left height + right height`. Track global max; return only the single best arm up.

Intuition: the longest path bends at some node using both arms, but only one arm can extend to its parent.

```
class Solution {
    int diameter = 0;
    public int diameterOfBinaryTree(TreeNode root) {
        dfs(root);
        return diameter;
    }
    private int dfs(TreeNode node) {
        if (node == null) return 0;
        int left = dfs(node.left);
        int right = dfs(node.right);
        diameter = Math.max(diameter, left + right);    // cross-node path
        return Math.max(left, right) + 1;              // single arm up
    }
}
```

Time $O(n)$ | **Space** $O(h)$

#124 Binary Tree Maximum Path Sum

Description: Maximum sum of any path between any two nodes. Values can be negative.

Key: clamp negative arms to 0 (`Math.max(0, ...)`). Global tracks the best `left + right + node.val`; return only `max(left, right) +`

`node.val` upward.

Intuition: a negative arm is worse than taking nothing, so clamp it to 0 before adding.

```
class Solution {
    int maxSum = Integer.MIN_VALUE;
    public int maxPathSum(TreeNode root) {
        dfs(root);
        return maxSum;
    }
    private int dfs(TreeNode node) {
        if (node == null) return 0;
        int left = Math.max(0, dfs(node.left));
        int right = Math.max(0, dfs(node.right));
        maxSum = Math.max(maxSum, left + right + node.val);
        return Math.max(left, right) + node.val;
    }
}
```

Time $O(n)$ | **Space** $O(h)$

#110 Balanced Binary Tree

Description: Is every node's left and right subtree within 1 height of each other?

Key: return -1 as a sentinel for "unbalanced"; short-circuit immediately on -1.

Intuition: fold "is it balanced" into the height return — a poisoned -1 bubbles up and stops the work.

```
class Solution {
    public boolean isBalanced(TreeNode root) {
        return dfs(root) != -1;
    }
    private int dfs(TreeNode node) {
        if (node == null) return 0;
        int left = dfs(node.left);
        if (left == -1) return -1;
        int right = dfs(node.right);
        if (right == -1) return -1;
        if (Math.abs(left - right) > 1) return -1;
        return Math.max(left, right) + 1;
    }
}
```

Time $O(n)$ | **Space** $O(h)$

#226 Invert Binary Tree

Description: Mirror a binary tree — swap left and right subtrees at every node.

Intuition: invert both children, then swap the pointers at this node.

```
class Solution {
    public TreeNode invertTree(TreeNode root) {
        if (root == null) return null;
        TreeNode left = invertTree(root.left);
        TreeNode right = invertTree(root.right);
        root.left = right;
        root.right = left;
        return root;
    }
}
```

Time $O(n)$ | **Space** $O(h)$

PROCESS NODE FIRST, PASS STATE DOWN (pre-order)

State is built on the way **down** and consumed at leaves. Use backtracking when state is a mutable collection.

#112 Path Sum

Description: Does any root-to-leaf path sum to `targetSum` ?

Intuition: subtract each node from the target on the way down; a leaf with remaining 0 is a hit.

```
class Solution {
    public boolean hasPathSum(TreeNode root, int targetSum) {
        return dfs(root, targetSum);
    }
    private boolean dfs(TreeNode node, int remaining) {
        if (node == null) return false;
        remaining -= node.val;
        if (node.left == null && node.right == null) return remaining == 0;
        return dfs(node.left, remaining) || dfs(node.right, remaining);
    }
}
```

Time $O(n)$ | **Space** $O(h)$

#113 Path Sum II

Description: Return all root-to-leaf paths that sum to `targetSum` .

Key: pass a mutable `path` list down. **Backtrack** (remove last element) after returning from each child.

Intuition: the same `path` list is reused for every branch, so undo your add as you unwind.

```
class Solution {
    public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
        List<List<Integer>> result = new ArrayList<>();
        dfs(root, targetSum, new ArrayList<>(), result);
        return result;
    }
    private void dfs(TreeNode node, int remaining, List<Integer> path, List<List<Integer>> result) {
        if (node == null) return;
        path.add(node.val);
        remaining -= node.val;
        if (node.left == null && node.right == null && remaining == 0)
            result.add(new ArrayList<>(path));
        dfs(node.left, remaining, path, result);
        dfs(node.right, remaining, path, result);
        path.remove(path.size() - 1); // backtrack
    }
}
```

Time $O(n)$ | **Space** $O(n)$ for output paths

#257 Binary Tree Paths

Description: Return all root-to-leaf paths as strings ("1->2->5").

Intuition: append to a shared `StringBuilder`, then truncate back to your entry length to undo.

```

class Solution {
    public List<String> binaryTreePaths(TreeNode root) {
        List<String> result = new ArrayList<>();
        dfs(root, new StringBuilder(), result);
        return result;
    }
    private void dfs(TreeNode node, StringBuilder path, List<String> result) {
        if (node == null) return;
        int len = path.length();
        if (len > 0) path.append("->");
        path.append(node.val);
        if (node.left == null && node.right == null)
            result.add(path.toString());
        dfs(node.left, path, result);
        dfs(node.right, path, result);
        path.setLength(len);    // backtrack to before this node
    }
}

```

Time $O(n)$ | **Space** $O(n)$ for output paths

#129 Sum Root to Leaf Numbers

Description: Each root-to-leaf path represents a number (e.g., $1 \rightarrow 2 \rightarrow 3 = 123$). Return the total sum.

Intuition: build the number digit by digit on the way down ($current * 10 + val$); add it up at each leaf.

```

class Solution {
    public int sumNumbers(TreeNode root) {
        return dfs(root, 0);
    }
    private int dfs(TreeNode node, int current) {
        if (node == null) return 0;
        current = current * 10 + node.val;
        if (node.left == null && node.right == null) return current;
        return dfs(node.left, current) + dfs(node.right, current);
    }
}

```

Time $O(n)$ | **Space** $O(h)$

543 vs 124 — Side by Side

Both use the global-variable pattern. One difference:

	#543 Diameter	#124 Max Path Sum
Global tracks:	left + right	left + right + node.val
Clamp negatives?	No (heights ≥ 0)	Yes ($\text{Math.max}(0, \dots)$)
Return up:	$\max(\text{left}, \text{right}) + 1$	$\max(\text{left}, \text{right}) + \text{node.val}$
Base case:	return 0	return 0

Part 2 — Graph DFS

Grid DFS (4-directional flood fill)

Mark visited by overwriting the cell value. Return the accumulated property (count, area) or void.

#200 Number of Islands

Description: Count the number of islands (connected groups of '1') in a binary grid.

Intuition: each unvisited land cell starts a new island; flood-fill sinks the whole island so it's counted once.


```

class Solution {
    public int numIslands(char[][] grid) {
        int count = 0;
        for (int r = 0; r < grid.length; r++)
            for (int c = 0; c < grid[0].length; c++)
                if (grid[r][c] == '1') { dfs(grid, r, c); count++; }
        return count;
    }
    private void dfs(char[][] grid, int r, int c) {
        if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return;
        if (grid[r][c] != '1') return;
        grid[r][c] = '0';    // mark visited
        dfs(grid, r+1, c); dfs(grid, r-1, c);
        dfs(grid, r, c+1); dfs(grid, r, c-1);
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#695 Max Area of Island

Description: Return the area of the largest island.

Key: same flood fill but DFS returns the area of the connected component.

Intuition: an island's area is 1 (this cell) plus the areas its four neighbors flood back.

```

class Solution {
    public int maxAreaOfIsland(int[][] grid) {
        int max = 0;
        for (int r = 0; r < grid.length; r++)
            for (int c = 0; c < grid[0].length; c++)
                max = Math.max(max, dfs(grid, r, c));
        return max;
    }
    private int dfs(int[][] grid, int r, int c) {
        if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return 0;
        if (grid[r][c] != 1) return 0;
        grid[r][c] = 0;
        return 1 + dfs(grid, r+1, c) + dfs(grid, r-1, c)
            + dfs(grid, r, c+1) + dfs(grid, r, c-1);
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

Graph DFS — 3-Color Cycle Detection

Three states: 0 = unvisited, 1 = in current DFS stack (visiting), 2 = fully processed (safe).

A back edge (reaching a node with state 1) means a cycle.

#207 Course Schedule

Description: Can you finish all courses given prerequisites? Return false if there is a cycle.

Intuition: re-entering a node still on the current DFS stack means you looped back to a prerequisite → cycle.

```

class Solution {
    int[] state;
    List<List<Integer>> graph;

    public boolean canFinish(int numCourses, int[][] prerequisites) {
        state = new int[numCourses];
        graph = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
        for (int[] p : prerequisites) graph.get(p[1]).add(p[0]);
        for (int i = 0; i < numCourses; i++)
            if (hasCycle(i)) return false;
        return true;
    }

    private boolean hasCycle(int node) {
        if (state[node] == 1) return true;    // back edge → cycle
        if (state[node] == 2) return false;  // already safe
        state[node] = 1;
        for (int neighbor : graph.get(node))
            if (hasCycle(neighbor)) return true;
        state[node] = 2;
        return false;
    }
}

```

Time $O(V+E)$ | **Space** $O(V+E)$

#210 Course Schedule II

Description: Return a valid order to take all courses. Return `[]` if a cycle exists.

Key: same 3-color DFS. Add node to `order` in **post-order** (after all neighbors done). Reverse at the end → topological order.

Intuition: a node finishes only after all it depends on do, so post-order reversed is a valid prerequisite order.

```

class Solution {
    int[] state;
    List<List<Integer>> graph;
    List<Integer> order;

    public int[] findOrder(int numCourses, int[][] prerequisites) {
        state = new int[numCourses];
        order = new ArrayList<>();
        graph = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
        for (int[] p : prerequisites) graph.get(p[1]).add(p[0]);
        for (int i = 0; i < numCourses; i++)
            if (!dfs(i)) return new int[0];
        Collections.reverse(order);
        return order.stream().mapToInt(Integer::intValue).toArray();
    }

    private boolean dfs(int node) {
        if (state[node] == 1) return false;    // cycle
        if (state[node] == 2) return true;     // already done
        state[node] = 1;
        for (int neighbor : graph.get(node))
            if (!dfs(neighbor)) return false;
        state[node] = 2;
        order.add(node);                      // post-order
        return true;
    }
}

```

Time $O(V+E)$ | **Space** $O(V+E)$

207 vs 210 — Side by Side

	#207 (boolean)	#210 (order)
DFS returns:	boolean (cycle found?)	boolean (cycle found?)
Extra action:	–	order.add(node) after neighbors
Post-process:	–	Collections.reverse(order)

Part 3 — Backtracking

Template: choose → recurse → undo.
start controls which elements are eligible at each level. current is the in-progress candidate.

```
// MENTAL MODEL: grow one candidate by picking an element, recurse, then undo to try the next branch.
private void backtrack(int start, List<Integer> current) {
    if (baseCase) { result.add(new ArrayList<>(current)); return; }
    for (int i = start; i < n; i++) {
        if (skipCondition) continue;
        current.add(nums[i]);           // choose
        backtrack(nextStart, current);  // nextStart = i (reuse) or i+1 (no reuse)
        current.remove(current.size() - 1); // undo
    }
}
```

Backtracking Decision Table

#	All orderings?	Reuse element?	Has duplicates?	Start index	Skip condition
46 Permutations	Yes — used[]	No	No	Loop 0..n, used[i]	used[i]
78 Subsets	No	No	No	start → i+1	None
39 Combination Sum	No	Yes	No	start → i	candidates[i] > remaining
40 Combination Sum II	No	No	Yes	start → i+1	i > start && nums[i] == nums[i-1]
131 Palindrome Part.	No	No	No	start → end	!isPalindrome(sub)

#46 Permutations

Description: All permutations of distinct integers.
Key: loop always from 0; use boolean[] used to avoid reusing elements within one permutation.
Intuition: order matters, so every position considers all unused elements (no start index).

```
class Solution {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, new boolean[nums.length], new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int[] nums, boolean[] used, List<Integer> current, List<List<Integer>> result) {
        if (current.size() == nums.length) { result.add(new ArrayList<>(current)); return; }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            used[i] = true;
            current.add(nums[i]);
            backtrack(nums, used, current, result);
            current.remove(current.size() - 1);
            used[i] = false;
        }
    }
}
```

Time O(n·n!) | Space O(n)

#78 Subsets

Description: All subsets (power set) of distinct integers.

Key: record at every node (not just leaves). Pass `i+1` — each element used at most once, order enforced by `start`.

Intuition: every prefix on the way down is itself a valid subset, so record at every node.

```
class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, 0, new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int[] nums, int start, List<Integer> current, List<List<Integer>> result) {
        result.add(new ArrayList<>(current)); // record every prefix (including empty)
        for (int i = start; i < nums.length; i++) {
            current.add(nums[i]);
            backtrack(nums, i + 1, current, result);
            current.remove(current.size() - 1);
        }
    }
}
```

Time $O(n \cdot 2^n)$ | **Space** $O(n)$

#39 Combination Sum

Description: All combinations summing to target. Same element may be used unlimited times. No duplicate combos.

Key: pass `i` (not `i+1`) to allow reuse. Sort + prune when `candidates[i] > remaining`.

Intuition: passing `i` (not `i+1`) lets you pick the same element again; `start` still forbids going backward, so no reordered dups.

```
class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        Arrays.sort(candidates);
        List<List<Integer>> result = new ArrayList<>();
        backtrack(candidates, 0, target, new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int[] candidates, int start, int remaining, List<Integer> current, List<List<Integer>> result) {
        if (remaining == 0) { result.add(new ArrayList<>(current)); return; }
        for (int i = start; i < candidates.length; i++) {
            if (candidates[i] > remaining) break; // pruning
            current.add(candidates[i]);
            backtrack(candidates, i, remaining - candidates[i], current, result); // i, not i+1
            current.remove(current.size() - 1);
        }
    }
}
```

Time $O(n \cdot 2^n)$ | **Space** $O(\text{target}/\text{min})$

#40 Combination Sum II

Description: Each element used at most once. Input may have duplicates. No duplicate combos in output.

Key: sort first. Skip `candidates[i] == candidates[i-1]` when `i > start` — avoids duplicate combos at the same recursion level. Pass `i+1` — no reuse.

Intuition: after sorting, equal values sit together; skipping a repeat at the *same level* prevents producing the same combo twice.

```

class Solution {
    public List<List<Integer>> combinationSum2(int[] candidates, int target) {
        Arrays.sort(candidates);
        List<List<Integer>> result = new ArrayList<>();
        backtrack(candidates, 0, target, new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int[] candidates, int start, int remaining, List<Integer> current, List<List<Integer>> result) {
        if (remaining == 0) { result.add(new ArrayList<>(current)); return; }
        for (int i = start; i < candidates.length; i++) {
            if (candidates[i] > remaining) break;
            if (i > start && candidates[i] == candidates[i - 1]) continue; // skip dup at same level
            current.add(candidates[i]);
            backtrack(candidates, i + 1, remaining - candidates[i], current, result); // i+1, no reuse
            current.remove(current.size() - 1);
        }
    }
}

```

Time $O(n \cdot 2^n)$ | **Space** $O(\text{target}/\text{min})$

#79 Word Search

Description: Given a board of characters, return true if the word exists as a connected path (no cell reused).

Key: mark cell as visited by overwriting, then restore after recursion (backtrack).

Intuition: the grid itself is the state — temporarily blank a cell so the same path can't reuse it, then restore on the way back.

```

class Solution {
    public boolean exist(char[][] board, String word) {
        for (int r = 0; r < board.length; r++)
            for (int c = 0; c < board[0].length; c++)
                if (dfs(board, word, r, c, 0)) return true;
        return false;
    }
    private boolean dfs(char[][] board, String word, int r, int c, int idx) {
        if (idx == word.length()) return true;
        if (r < 0 || r >= board.length || c < 0 || c >= board[0].length) return false;
        if (board[r][c] != word.charAt(idx)) return false;
        char temp = board[r][c];
        board[r][c] = '#'; // mark visited
        boolean found = dfs(board, word, r+1, c, idx+1) || dfs(board, word, r-1, c, idx+1)
            || dfs(board, word, r, c+1, idx+1) || dfs(board, word, r, c-1, idx+1);
        board[r][c] = temp; // restore (backtrack)
        return found;
    }
}

```

Time $O(m \cdot n \cdot 4^L)$ where L = word length | **Space** $O(L)$

#131 Palindrome Partitioning

Description: Partition string `s` such that every substring is a palindrome. Return all such partitions.

Intuition: try every prefix as the first cut; only recurse on the rest when that prefix is a palindrome.

```

class Solution {
    public List<List<String>> partition(String s) {
        List<List<String>> result = new ArrayList<>();
        backtrack(s, 0, new ArrayList<>(), result);
        return result;
    }
    private void backtrack(String s, int start, List<String> current, List<List<String>> result) {
        if (start == s.length()) { result.add(new ArrayList<>(current)); return; }
        for (int end = start + 1; end <= s.length(); end++) {
            String sub = s.substring(start, end);
            if (!isPalindrome(sub)) continue;
            current.add(sub);
            backtrack(s, end, current, result);
            current.remove(current.size() - 1);
        }
    }
    private boolean isPalindrome(String s) {
        int i = 0, j = s.length() - 1;
        while (i < j) if (s.charAt(i++) != s.charAt(j--)) return false;
        return true;
    }
}

```

Time $O(n \cdot 2^n)$ | **Space** $O(n)$

39 vs 40 — The One Difference

	#39 (reuse OK)	#40 (no reuse, has dups)
Sort:	Yes (pruning)	Yes (both pruning and dup-skip)
Recurse with:	i (same index)	i+1 (next index)
Skip dup line:	—	if (i > start && nums[i] == nums[i-1]) continue

Tree DFS Summary

Pattern	When to use	Answer location
Return up	Combine children at node	return value bubbles up
Pass down	Accumulate state toward leaves	check at leaf
Global variable	Path spans across a node (not up)	int/long field, updated inside dfs

Part — BST (Binary Search Tree) Problems

BST key property: `left subtree values < node.val < right subtree values`. This allows $O(h)$ search/insert/delete by binary-searching the tree.

BST Template: Pass Bounds Down

```

// Validate BST by passing min/max constraints down the tree
boolean validate(TreeNode node, long min, long max) {
    if (node == null) return true;
    if (node.val <= min || node.val >= max) return false; // ← bounds check
    return validate(node.left, min, node.val)           // left: tighten max
        && validate(node.right, node.val, max);         // right: tighten min
}

// Call with: validate(root, Long.MIN_VALUE, Long.MAX_VALUE)
// Use Long to handle Integer.MIN_VALUE / Integer.MAX_VALUE edge cases

```

BST Template: Inorder Traversal = Sorted Order

```
// BST inorder (left → node → right) visits nodes in ascending order
void inorder(TreeNode node) {
    if (node == null) return;
    inorder(node.left);
    // process node (kth smallest: increment counter, return when k)
    inorder(node.right);
}
```

BST Template: Binary Search on Tree

```
// Navigate BST like binary search: go left or right based on comparison
TreeNode search(TreeNode root, int target) {
    while (root != null) {
        if (root.val == target) {
            return root;
        } else if (root.val < target) {
            root = root.right;
        } else {
            root = root.left;
        }
    }
    return null;
}
```

#98 Validate Binary Search Tree

Description: Determine if a binary tree is a valid BST. Each node's value must be strictly greater than all values in its left subtree and strictly less than all values in its right subtree.

Variation: Pass min/max bounds down — each call tightens the valid range. Use `long` to handle `Integer.MIN_VALUE` and `Integer.MAX_VALUE` edge cases.

```
class Solution {
    public boolean isValidBST(TreeNode root) {
        return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    private boolean validate(TreeNode node, long min, long max) {
        if (node == null) return true;
        if (node.val <= min || node.val >= max) return false; // ← VARIATION: bounds constraint
        return validate(node.left, min, node.val)           // ← VARIATION: tighten max going left
            && validate(node.right, node.val, max);           // ← VARIATION: tighten min going right
    }
}
```

Time $O(n)$ | **Space** $O(h)$

#230 Kth Smallest Element in a BST

Description: Find the kth smallest element in a BST (1-indexed).

Variation: BST inorder traversal visits nodes in ascending order. Count nodes visited; when count reaches k, record the answer and stop early.

```

class Solution {
    private int count = 0, result = 0;

    public int kthSmallest(TreeNode root, int k) {
        inorder(root, k);
        return result;
    }

    private void inorder(TreeNode node, int k) {
        if (node == null) return;
        inorder(node.left, k);
        if (++count == k) { result = node.val; return; } // ← VARIATION: stop at kth node
        inorder(node.right, k);
    }
}

```

Time O(k) average, O(n) worst | **Space** O(h)

#270 Closest Binary Search Tree Value

Description: Given a BST and a `target` (double), return the value in the BST closest to target.

Variation: Use BST structure to navigate in O(h). At each node, compare distance to closest seen so far, then binary-search left or right.

```

class Solution {
    public int closestValue(TreeNode root, double target) {
        int closest = root.val;
        while (root != null) {
            if (Math.abs(root.val - target) < Math.abs(closest - target))
                closest = root.val; // ← VARIATION: track closest during search
            root = root.val < target ? root.right : root.left; // ← VARIATION: binary search
        }
        return closest;
    }
}

```

Time O(h) | **Space** O(1)

#285 Inorder Successor in BST

Description: Given a node `p` in a BST, return its inorder successor (the smallest node with value greater than `p.val`). Return null if no such node exists.

Variation: Navigate the BST. When `root.val > p.val`, this root is a candidate — save it and go left (to find a smaller valid candidate). When `root.val <= p.val`, go right (must find something larger).

```

class Solution {
    public TreeNode inorderSuccessor(TreeNode root, TreeNode p) {
        TreeNode result = null;
        while (root != null) {
            if (root.val > p.val) {
                result = root; // ← VARIATION: save as candidate, then search left for closer
                root = root.left;
            } else {
                root = root.right; // ← VARIATION: must go right to find something > p
            }
        }
        return result;
    }
}

```

Time O(h) | **Space** O(1)

#687 Longest Univalue Path

Description: Return the length of the longest path in a binary tree where every node along the path has the same value. The path does not need to pass through the root.

Variation: Post-order DFS. At each node, extend the left or right path only if the child's value matches. The path through the current node = leftPath + rightPath. Return the max single-direction extension upward.

```
class Solution {
    private int max = 0;

    public int longestUnivaluePath(TreeNode root) {
        dfs(root);
        return max;
    }

    private int dfs(TreeNode node) {
        if (node == null) return 0;
        int left = dfs(node.left);
        int right = dfs(node.right);
        int leftPath = (node.left != null && node.left.val == node.val) ? left + 1 : 0; // ← VARIATION: only extend if values match
        int rightPath = (node.right != null && node.right.val == node.val) ? right + 1 : 0; // ← VARIATION: conditional extension
        max = Math.max(max, leftPath + rightPath); // ← path through this node
        return Math.max(leftPath, rightPath); // ← return max single direction up
    }
}
```

Time $O(n)$ | **Space** $O(h)$

#37 Sudoku Solver

Description: Fill a partially filled 9×9 Sudoku so every row, column, and 3×3 box contains digits 1-9. **Variation:** backtracking that tries digits 1-9 in each empty cell, validating against row, column, and box constraints before recursing.

```
class Solution {
    public void solveSudoku(char[][] board) { solve(board); }
    private boolean solve(char[][] board) {
        for (int i = 0; i < 9; i++)
            for (int j = 0; j < 9; j++)
                if (board[i][j] == '.') {
                    for (char c = '1'; c <= '9'; c++) {
                        if (isValid(board, i, j, c)) { // ← VARIATION: validate row/col/box
                            board[i][j] = c;
                            if (solve(board)) return true;
                            board[i][j] = '.'; // backtrack
                        }
                    }
                    return false; // no digit fits → dead end
                }
        return true; // all cells filled
    }
    private boolean isValid(char[][] board, int row, int col, char c) {
        for (int i = 0; i < 9; i++) {
            if (board[row][i] == c) return false; // row
            if (board[i][col] == c) return false; // column
            if (board[3*(row/3) + i/3][3*(col/3) + i%3] == c) return false; // ← VARIATION: 3x3 box
        }
        return true;
    }
}
```

Time $O(9^m)$ m =empty cells | **Space** $O(1)$

#47 Permutations II

Description: Return all unique permutations of an array that may contain duplicates. **Variation:** sort first; use a `used[]` array; skip a value if the previous equal value at the same tree level has not been used (prevents duplicate permutations).

```
class Solution {
    public List<List<Integer>> permuteUnique(int[] nums) {
        Arrays.sort(nums); // ← VARIATION: sort to group duplicates
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, new boolean[nums.length], new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int[] nums, boolean[] used, List<Integer> path, List<List<Integer>> result) {
        if (path.size() == nums.length) { result.add(new ArrayList<>(path)); return; }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            if (i > 0 && nums[i] == nums[i-1] && !used[i-1]) continue; // ← VARIATION: skip duplicate at same level
            used[i] = true; path.add(nums[i]);
            backtrack(nums, used, path, result);
            used[i] = false; path.remove(path.size() - 1);
        }
    }
}
```

Time $O(n \cdot n!)$ | **Space** $O(n)$

#51 N-Queens

Description: Place n queens on an $n \times n$ board so no two attack each other; return all distinct solutions. **Variation:** backtrack row by row; track occupied columns and both diagonals. Cells on the same $\backslash$ diagonal share $r-c$; cells on the same $/$ diagonal share $r+c$.

```
class Solution {
    public List<List<String>> solveNQueens(int n) {
        List<List<String>> result = new ArrayList<>();
        Set<Integer> cols = new HashSet<>(), diag1 = new HashSet<>(), diag2 = new HashSet<>();
        backtrack(0, n, new int[n], cols, diag1, diag2, result);
        return result;
    }
    private void backtrack(int row, int n, int[] queens, Set<Integer> cols,
        Set<Integer> diag1, Set<Integer> diag2, List<List<String>> result) {
        if (row == n) { result.add(build(queens, n)); return; }
        for (int c = 0; c < n; c++) {
            if (cols.contains(c) || diag1.contains(row - c) || diag2.contains(row + c)) continue; // ← VARIATION: diagonal sets
            queens[row] = c;
            cols.add(c); diag1.add(row - c); diag2.add(row + c);
            backtrack(row + 1, n, queens, cols, diag1, diag2, result);
            cols.remove(c); diag1.remove(row - c); diag2.remove(row + c);
        }
    }
    private List<String> build(int[] queens, int n) {
        List<String> board = new ArrayList<>();
        for (int r = 0; r < n; r++) {
            char[] row = new char[n];
            Arrays.fill(row, '.');
            row[queens[r]] = 'Q';
            board.add(new String(row));
        }
        return board;
    }
}
```

Time $O(n!)$ | **Space** $O(n)$

#60 Permutation Sequence

Description: Return the kth permutation (1-indexed) of the sequence 1..n. **Variation:** no backtracking — use the factorial number system. Each position's digit is determined directly by $(k-1) / (\text{remaining}-1)!$.

```
class Solution {
    public String getPermutation(int n, int k) {
        List<Integer> digits = new ArrayList<>();
        int[] factorial = new int[n + 1];
        factorial[0] = 1;
        for (int i = 1; i <= n; i++) { factorial[i] = factorial[i-1] * i; digits.add(i); }
        k--; // ← VARIATION: convert to 0-indexed
        StringBuilder result = new StringBuilder();
        for (int i = n; i >= 1; i--) {
            int index = k / factorial[i-1]; // ← VARIATION: direct digit selection
            k %= factorial[i-1];
            result.append(digits.remove(index));
        }
        return result.toString();
    }
}
```

Time $O(n^2)$ | **Space** $O(n)$

#90 Subsets II

Description: Return all possible unique subsets of an array that may contain duplicates. **Variation:** sort first; within a recursion level, skip a value equal to its predecessor to avoid duplicate subsets.

```
class Solution {
    public List<List<Integer>> subsetsWithDup(int[] nums) {
        Arrays.sort(nums); // ← VARIATION: sort to group duplicates
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, 0, new ArrayList<>(), result);
        return result;
    }

    private void backtrack(int[] nums, int start, List<Integer> path, List<List<Integer>> result) {
        result.add(new ArrayList<>(path));
        for (int i = start; i < nums.length; i++) {
            if (i > start && nums[i] == nums[i-1]) continue; // ← VARIATION: skip duplicate at same depth
            path.add(nums[i]);
            backtrack(nums, i + 1, path, result);
            path.remove(path.size() - 1);
        }
    }
}
```

Time $O(n \cdot 2^n)$ | **Space** $O(n)$

#216 Combination Sum III

Description: Find all combinations of k numbers from 1-9 (each used at most once) that sum to n. **Variation:** standard combination backtracking with start index `i+1` (no reuse), but with two stopping constraints — count and remaining sum.

```

class Solution {
    public List<List<Integer>> combinationSum3(int k, int n) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(k, n, 1, new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int k, int remain, int start, List<Integer> path, List<List<Integer>> result) {
        if (path.size() == k && remain == 0) { result.add(new ArrayList<>(path)); return; } // ← VARIATION: two constraints
        if (path.size() == k || remain < 0) return;
        for (int i = start; i <= 9; i++) {
            path.add(i);
            backtrack(k, remain - i, i + 1, path, result); // i+1: no reuse
            path.remove(path.size() - 1);
        }
    }
}

```

Time $O(C(9,k))$ | **Space** $O(k)$

#549 Binary Tree Longest Consecutive Sequence II

Description: Find the length of the longest consecutive path in a binary tree. The path can be increasing or decreasing and may go through a node (child-parent-child). **Variation:** post-order DFS returns a pair `{increasing, decreasing}` for each node. A path through the node combines an increasing run from one child with a decreasing run from the other.

```

class Solution {
    private int max = 0;
    public int longestConsecutive(TreeNode root) {
        dfs(root);
        return max;
    }
    private int[] dfs(TreeNode node) { // returns {inc, dec}
        if (node == null) return new int[]{0, 0};
        int inc = 1, dec = 1;
        int[] left = dfs(node.left), right = dfs(node.right);
        if (node.left != null) {
            if (node.left.val + 1 == node.val) inc = left[0] + 1; // ← VARIATION: increasing run
            else if (node.left.val - 1 == node.val) dec = left[1] + 1; // ← VARIATION: decreasing run
        }
        if (node.right != null) {
            if (node.right.val + 1 == node.val) inc = Math.max(inc, right[0] + 1);
            else if (node.right.val - 1 == node.val) dec = Math.max(dec, right[1] + 1);
        }
        max = Math.max(max, inc + dec - 1); // ← VARIATION: combine both directions through node
        return new int[]{inc, dec};
    }
}

```

Time $O(n)$ | **Space** $O(h)$